

SN	Course Code:	Course Name: Data Structures and Algorithm	L	T	P	S	C	CH	Course Category	Course Type
1	25CSH-103	Course Coordinator: Dr. Sheenam	2	0	4	0	4	6	PC	Hybrid
Pre-Requisites:		Basic knowledge of Computer Programming								
Co-Requisite		Nil								
Anti-Requisite		Nil								

A. COURSE DESCRIPTION:

This course focuses on the design, implementation, and analysis of fundamental data structures using C/C++ programming languages. It covers widely used data structures such as dynamic arrays, linked lists, stacks, and queues, which are essential for solving various computational problems. Through hands-on programming assignments, students will learn how these data structures are implemented in C/C++, gaining practical experience and a deeper understanding of how built-in data structures function internally and what performance behaviour to expect from them.

B. COURSE OBJECTIVES:

- To understand the concept of data structure and various operations on them.
- To understand the properties of various data structures and able to identify the strengths and weaknesses of different data structures.
- To design and employ appropriate data structures for solving computing problems.
- To analyze and compare the efficiency of algorithms and learn to design efficient algorithms for solving computing problems.

C. COURSE OUTCOMES

CO No	Statement	Performance Indicator	Student Outcome Indicator (ABET)	Level of Learning (Highest BT Level)	Target Attainment
CO1	To analyse and compare the efficiency and properties of various data structures.	SO1	1a	EASY	
CO2	To identify strength and weaknesses of various data structures.	SO1,SO7	1a, 7a,7b	MEDIUM	
CO3	To design and employ appropriate data structures for solving computing problems.	SO7	7b	HARD	
CO4	To possess the ability to design efficient algorithms for solving computing problems.	SO1	1a,1b,1c,1d	MEDIUM	
CO5	To demonstrate an ability to apply design and development principles in the construction of software systems of varying complexity.	SO1,SO7	1c,1d,7b	HARD	

D. SYLLABUS

Unit-1	Fundamental of Data structure	Contact Hours: 10 H
Chapter 1.1 Introduction to Data Structure	Concept of Data and Information, Introduction to Data Structures, Types of data structure: Linear and non-linear data structures, Operations on Data Structures, Time and Space complexity.	
Chapter 1.2 Arrays	Basic terminology, Linear arrays and their representation, Traversing Linear Array, Insertion & Deletion in arrays, 2D Array Declaration, Initialization, Operations on 2-D Array, Arrays and pointers.	
Chapter 1.3 Searching	Linear Search, Binary Search, Complexity analysis of each algorithm.	
Self-Study topics	Analysis of Time and Space complexity	
Unit-2	Sorting and Linked List	Contact Hours: 10 H
Chapter 2.1 Sorting	Bubble sort, Insertion sort, Selection sort, Merge Sort and Quick Sort complexity analysis of each algorithm.	
Chapter 2.2 Linked List	Linear linked list, Representation of Linked Lists in Memory, traversing a Linked List, Searching in a Linked List, Insertion & Deletion from Linked List, Doubly Linked List, Circular Linked List, Complexity Analysis of each operation, Application of Linked lists.	
Self-Study topics	Memory Allocation in Linked Lists	
Unit-3	Stack and Queues	Contact Hours: 10 H
Chapter 3.1 Stacks	Basic Terminology, Sequential and Linked Representations, Operations on Stacks: PUSH & POP, Application of Stacks: Evaluation of Postfix Expressions, Conversion from Prefix to Postfix Representation, Parenthesis Matching	
Chapter 3.2 Queues	Linear Queue, Sequential and Linked Representation of Linear Queue, Circular Queue, Operations on Queue, Introduction to priority Queue.	
Self-Study topics	Real world applications of Stack and Queues	

E. SYLLABUS PRACTICAL

Sr. No.	Practical Name	CO Mapping
UNIT-1	FUNDAMENTAL OF DATA STRUCTURE	Contact Hours: 20
Topic Name	Applications of Arrays	
1	- Program to demonstrate storing and retrieving data using 1D and 2D arrays. - Design a program to remove duplicate elements from an array of size 10. - Implement a function to reverse an array in-place, without using an additional array. - Introduction to vectors and CRUD operations in vectors. - Develop a program to merge two sorted arrays into a single sorted array. - Demonstrate a program to find the maximum and minimum elements in an array. - Create a program to separate even and odd numbers within an array.	CO1, CO2
	Demonstrate the working of Searching techniques	
2	- Implement linear search to find a given element in an array. - Apply binary search to locate a specific element in a sorted array (ascending	CO1, CO2

	order). Develop a program to count the occurrences of a target element entered by the user (lower bound and upper bound).	
UNIT -2	SORTING AND LINKED LIST	Contact Hours: 20
	Real World problems on Sorting techniques	
3	- Implement bubble sort to arrange an array in ascending order. - Demonstrate the working of insertion sort for sorting an array. - Program to sort an array of integers in ascending order using selection sort. - Write a program to sort an element using quick sort and analyse its performance. - Write a recursive program to implement mergesort.	CO2, CO3
	Applications of Linked List	
4	- Create a menu-driven program to manage a sorted singly linked list with the following operations (each implemented as a separate function): - Create a linked list. - Insert a new element while maintaining order. - Delete an existing element. - Search for an element. - Display all elements of the linked list. - Implement a menu-driven program for performing operations on a doubly linked list, including: - Creation of a doubly linked list. - Insertion of an element. - Deletion of an element. - Traversal and display of elements	CO2, CO3
UNIT -3	STACK AND QUEUES	Contact Hours: 20
	Operations on Stack	
5	- Develop a menu-driven program to perform stack operations using an array-based implementation: - Push an element onto the stack. - Pop an element from the stack. - Handle stack overflow and underflow conditions. - Display the current state of the stack. - Write a program to demonstrate the use of stack (implemented using linear array) in converting arithmetic expression from infix notation to postfix notation.	CO4, CO5
	Application of Queues	
6	- Implement a menu-driven program to perform operations on a Linear Queue using an array: - Insertion (Enqueue) - Deletion (Dequeue) - Display the elements - Check for Overflow and Underflow conditions - Develop a program to implement a Linear Queue using pointers (Linked List implementation) and support the following operations:	CO4, CO5

	1) Insertion (Enqueue) at the rear 2) Deletion (Dequeue) from the front 3) Display all queue elements 4) Check if the queue is empty or full	
--	---	--

F. EXPERIENTIAL LEARNING COMPONENTS [[Default Course Projects / Case Studies / Field Projects]

1. Write a code to implement the Tower of Hanoi problem using recursion.
2. Linked List Reverse Printing Challenge. Using programming techniques you know, write the cleanest possible function you can think of to print a singly linked list in reverse. The format for the node should be a struct containing an integer value, val, and a next pointer to the following node.
3. A hotel maintains room booking records using an array. The system should include modular functions to:
 - a) Add a new booking (Mark room as booked)
 - b) Search for an available room
 - c) Display all available rooms
 - d) Cancel a booking (Mark room as available)
4. A warehouse keeps track of product stock using an array. The system should include modular functions to:
 - a) Add new products to stock
 - b) Update stock after a sale
 - c) Display current stock levels
 - d) Remove a product from inventor
5. A graphics software processes images as $n \times n$ matrices. The system should include modular functions to:
 - a) Accept user input for a matrix
 - b) Rotate the matrix by 90 degrees
 - c) Display the rotated matrix
 - d) Reset the matrix to its original state
6. A social media platform stores user friendship as a linked list. The system should include modular functions to:
 - a) Add a new friend connection
 - b) Search for mutual friends
 - c) Display all friend connections for a user
 - d) Remove a friend from the list
7. A text editor implements an undo operation using a stack. The system should include modular functions to:
 - a) Add text to the editor
 - b) Undo the last action
 - c) Redo the last undone action
 - d) Display the current text
8. A compiler checks code syntax using a stack to validate bracket sequences. The system should include modular functions to:
 - a) Accept an input expression
 - b) Check if brackets are balanced
 - c) Display a validation result
 - d) Reset the validation check
9. A calculator evaluates postfix expressions using a stack. The system should include modular

functions to:

- a) Accept a postfix expression
 - b) Evaluate the expression using a stack
 - c) Display the result
 - d) Reset the calculator
10. A restaurant processes food orders using a queue. The system should include modular functions to:
- a) Add a new customer order
 - b) Serve the next order in the queue
 - c) Display all pending orders
 - d) Remove a cancelled order
11. A printer manages print jobs based on priority using a priority queue. The system should include modular functions to
- a) Add a new print job with priority
 - b) Process the highest-priority job
 - c) Display the print queue
 - d) Remove a job from the queue
12. A ticket counter processes customers in a queue. The system should include modular functions to:
- a) Add a new customer to the queue
 - b) Serve the next customer
 - c) Display the waiting queue
 - d) Remove a customer from the queue
13. A programming tool converts infix expressions to postfix and prefix using stacks. The system should include modular functions to:
1. Accept an infix expression
 2. Convert it to postfix notation
 3. Convert it to prefix notation
 4. Display both conversions
14. A math software processes expressions using binary expression trees. The system should include modular functions to
1. Construct an expression tree from an infix expression
 2. Convert the tree to a postfix expression
 3. Evaluate the expression using the tree
 4. Delete the expression tree
15. A web browser maintains browsing history using a stack. The system should include modular functions to
1. Record a new visited page
 2. Navigate back to the previous page
 3. Display browsing history
 4. Clear browsing history

G. TEXTBOOKS/ REFERENCE BOOKS

i. TEXTBOOKS

T1: Seymour Lipschutz "Data Structures", Schaum's Outline Series, Tata McGraw-Hill, 2000.

T2: Reema Thareja – "Data Structures Using C", Oxford University Press, 2nd Edition, 2014.

T3: Yashavant P. Kanetkar – "Data Structures Through C", BPB Publications, 2008.

T4: Debasis Samanta – "Classic Data Structures", PHI Learning, 2nd Edition, 2009.

T5: K.S. Easwarakumar – "Data Structures Using C", Pearson Education, 2017.

ii. REFERENCE BOOKS

R1: Mark Allen Weiss – "Data Structures and Algorithm Analysis in C", Pearson, 2nd Edition, 1997.

R2: Ellis Horowitz, Sartaj Sahni, Susan Anderson-Freed – "Fundamentals of Data Structures in C", Universities Press, 2nd Edition, 2008.

R3: Tenenbaum, Langsam, & Augenstein – "Data Structures Using C", Pearson Education, 2nd Edition, 2009.

R4: Narasimha Karumanchi – "Data Structures and Algorithms Made Easy in C", CareerMonk Publications, 2016.

R5: Rajesh K. Shukla – "Data Structures Using C & C++", Wiley India, 2016.

H. ASSESSMENT PATTERN

The performance of students is evaluated as follows:

Components	Theory		Practical	
	Continuous Assessment (CAE)	Internal Examination (SEE)	Continuous Internal Assessment (CAE)	Semester End Examination (SEE)
Marks	50	50	60	40
Total Marks	100		100	

Internal & External Exam Evaluation Components (Theory):

S No.	Direct Evaluation Instruments	Weightage of Actual Conduct	Assessment Tool	Skills Evaluated	Employability Linkage (Industry Needs)	Frequency of Task	Final Weightage in Internal / External Evaluation	Assessment Rubrics
1	Assignment	12	Offline	Problem Solving, Algorithm Design, Code Efficiency	Software Developer, competitive Programmer	2	12	NA

2	MST 1 / MST2	20	Online coding Platform	Problem Solving, Algorithm Design, Code Efficiency	Software Developer, competitive Programmer	2	10(Each MST)	NA
3	Case Study	10	Offline in the form of PPT	Problem Solving, Algorithm Design, Code Efficiency	Software Developer, competitive Programmer	1	10	NA
4	Surprise Test/ Quiz	6	MCQ based on Online coding Platform	Problem Solving, Algorithm Design, Code Efficiency	Software Developer, competitive Programmer	2	6	NA
5	Attendance	2	-	-	-	-	2	NA
6	EST	50	-	-	-	1	50	NA

Internal & External Exam Evaluation Components (Practical):

S No.	Direct Evaluation Instruments	Weightage of actual conduct	Assessment Tool	Skills Evaluated	Employability Linkage (Industry Needs)	Frequency of Task	Final Weightage in Internal Assessment /External Evaluation	Assessment Rubrics
1	Unit-1 Practical Evaluation	30 marks	Online coding Platform	Problem Solving, Algorithm Design, Code Efficiency	Software Developer, competitive Programmer	1	15	Conduct- 12 marks Viva- 10 marks Contiguous Evaluations - 8

								marks
2	Unit-2 Practical Evaluation	30 marks	Online coding Platform	Problem Solving, Algorithm Design, Code Efficiency	Software Developer, competitive Programmer	1	15	Conduct- 12 marks Viva- 10 marks Contiguous Evaluations - 8 marks
3	Unit-3 Practical Evaluation/ MiniProject	30 marks	Online coding Platform	Problem Solving, Algorithm Design, Code Efficiency	Software Developer, competitive Programmer	1	15	Conduct- 12 marks Viva- 10 marks Contiguous Evaluations - 8 marks
4	Lab MST	30 marks	Online coding Platform	Problem Solving, Algorithm Design, Code Efficiency	Software Developer, competitive Programmer	1	15	Conduct- 12 marks Viva- 10 marks Contiguous Evaluations - 8 marks
5	External Practical Exam	40 Marks	Online coding Platform	Problem Solving, Algorithm Design, Code Efficiency	Software Developer, competitive Programmer	1	40	Conduct- 20 marks Viva - 10marks Worksheet- 10 marks

I. CO-PO MAPPING

Course Outcome	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10
CO1	1	1	2	3	0	0	0	0	0	0
CO2	1	1	2	3	2	0	0	0	1	0
CO3	3	1	2	3	0	0	0	0	0	0
CO4	3	1	2	3	0	0	0	0	0	0
CO5	3	2	2	3	0	0	0	0	2	0

[illegible]

CO2	0	1	0	0	1	0	0	0	0	0
CO3	0	0	0	0	0	0	0	0	1	1
CO4	0	0	0	0	0	0	0	0	1	1
CO5	0	0	0	0	0	0	0	3	2	2

CO PO correlation matrix of each subject to be mapped with
High correlation (3); Medium correlation (2); Low correlation (1)